

ОПИ: тестирование, сборка и build tools

Июнь 2026

Содержание

1	Короткие ответы к вопросам	4
1.1	Тестирование ПО	4
1.2	JUnit	4
1.3	Make и Makefile	4
1.4	GNU Autotools	5
1.5	Ant	5
1.6	Maven	5
1.7	Gradle	6
2	Блок 1. Основы тестирования	7
2.1	Термины: ошибка, дефект, отказ	7
2.2	Цели тестирования	7
2.3	Верификация и валидация	7
2.4	Как проверять соответствие реализации требованиям	7
2.5	Тестовый случай и тестовый сценарий	8
2.6	Статическое и динамическое тестирование	8
2.7	Ручное, автоматизированное и регрессионное тестирование	9
3	Блок 2. Покрытие тестами	10
3.1	Что такое тестовое покрытие	10
3.2	Покрытие по методу чёрного ящика	10
3.2.1	Эквивалентное разбиение	10
3.2.2	Анализ граничных значений	10
3.2.3	Таблицы решений	10
3.2.4	Таблицы состояний и переходов	10
3.2.5	Сценарии использования	11
3.3	Покрытие по методу белого ящика	11
3.4	Дополнительные виды покрытия	11
4	Блок 3. Уровни тестирования и V-модель	12
4.1	Модульное тестирование	12
4.2	Интеграционное тестирование	12
4.3	Стратегии интеграции	12
4.4	Функциональное тестирование	12
4.5	Системное и приёмочное тестирование	13
4.6	Нефункциональное и нагрузочное тестирование	13
4.7	V-модель	13

5	Блок 4. JUnit	14
5.1	Идея JUnit	14
5.2	Основные аннотации JUnit 4	14
5.3	Аналоги в JUnit Jupiter	14
5.4	Assert-методы	14
5.5	Минимальный пример	15
6	Блок 5. Зачем нужны системы сборки	16
6.1	Проблемы ручной сборки	16
6.2	Что делают build tools	16
6.3	Императивный и декларативный подход	16
7	Блок 6. Make и Makefile	17
7.1	Идея Make	17
7.2	Дефолтная цель	17
7.3	Обычные переменные Make	18
7.4	Автоматические переменные	18
7.5	Плюсы и минусы Make	18
8	Блок 7. GNU Autotools	19
8.1	Назначение	19
8.2	Основные утилиты и роли	19
8.3	Файлы проекта	19
8.4	Процесс для разработчика	19
8.5	Что делает <code>./configure</code>	20
8.6	Плюсы и минусы Autotools	20
9	Блок 8. Apache Ant	21
9.1	Идея Ant	21
9.2	Project, target, task	21
9.3	Default target	21
9.4	Свойства Ant	21
9.5	Типичные задачи Ant	22
9.6	CLI-флаги	22
9.7	<code>antcall</code>	22
9.8	<code>subant</code>	22
9.9	Ivy	23
10	Блок 9. Maven	24
10.1	Идея Maven	24
10.2	Чем Maven лучше Ant	24
10.3	GAV	24
10.4	Scope зависимостей	25
10.5	Жизненный цикл Maven	25
10.6	Плагины	25
10.7	Профили	26
11	Блок 10. Gradle	27
11.1	Идея Gradle	27
11.2	Пример Gradle build script	27
11.3	Чем Gradle отличается от Maven	27

11.4	Жизненный цикл Gradle	27
11.5	Преимущества Gradle	28
11.6	Минусы Gradle	28
12	Приложение: лабораторная 2	28
12.1	Тестирование ПО	28
12.2	Цели тестирования	29
12.3	Ошибка, дефект и отказ	29
12.4	Виды тестирования	29
12.4.1	По уровню тестирования	29
12.4.2	По способу выполнения	29
12.4.3	По знанию внутреннего устройства	29
12.4.4	По цели тестирования	30
12.5	Модульное тестирование	30
12.6	Принцип FIRST	30
12.7	Структура AAA	30
12.8	Given–When–Then	31
12.9	Что проверять в модульных тестах	31
12.10	Тестовые двойники	31
12.11	Test-Driven Development	31
12.12	JUnit	32
12.13	Аннотации JUnit	32
12.14	Assertions в JUnit	32
12.15	Системы автоматической сборки	33
12.16	Принципы работы систем сборки	33
12.17	Основные системы сборки	33
12.18	Make	33
12.19	Ant	34
12.20	Типичные Ant tasks	34
12.21	Maven: что важно добавить к концу лаб2	35
12.22	Gradle: что важно добавить к концу лаб2	35

1 Короткие ответы к вопросам

1.1 Тестирование ПО

Что такое уровни тестирования? Уровни тестирования — это последовательные уровни проверки ПО, соответствующие разной крупности объекта тестирования: модульное/компонентное, интеграционное, системное, приёмочное. В современной ISTQB-терминологии также явно разделяют компонентное интеграционное и системное интеграционное тестирование. В лекции это объясняется через V-модель: после написания кода проверяют модули, затем взаимодействие модулей, затем всю систему, затем готовность продукта к приёмке заказчиком.

Как проверить соответствие реализации требованиям? Нужно связать требования с тестами и результатами: из требований выделить проверяемые условия, построить тестовые случаи с заранее известным ожидаемым результатом, выполнить их в целевом окружении, сравнить фактический результат с ожидаемым и зафиксировать отклонения как дефекты. Это называется верификацией реализации относительно требований. Дополнительно применяют статические техники: ревью требований, дизайна, кода, планов и тестовых сценариев.

Какие виды покрытия тестами знаете? Покрытие требований, функциональное покрытие, покрытие сценариев использования, покрытие классов эквивалентности, граничных значений, таблиц решений, состояний и переходов, а также структурное покрытие кода: операторов/строк, ветвей/решений, условий, путей, циклов. Важная мысль лекции: полное покрытие почти всегда недостижимо, поэтому нужно выбирать достаточное покрытие по рискам, требованиям и стоимости.

Что такое V-модель тестирования? V-модель связывает стадии разработки со стадиями тестирования. Требования сопоставляются с приёмочным и системным тестированием, архитектурное проектирование — с интеграционным, детальное проектирование и код — с модульным. Левая ветвь “V” описывает уточнение требований и проектирование, нижняя точка — кодирование, правая ветвь — последовательную проверку результата.

1.2 JUnit

Какие основные аннотации JUnit знаете? Для JUnit 4 из лекции: @Test, @Before, @After, @BeforeClass, @AfterClass, @Ignore, @Rule, @ClassRule, @RunWith. В JUnit Jupiter/JUnit 5+: @Test, @BeforeEach, @AfterEach, @BeforeAll, @AfterAll, @Disabled, @ParameterizedTest, @RepeatedTest, @Nested, @Tag, @ExtendWith, @Timeout.

Какие assert-методы используете? assertEquals, assertNotEquals, assertTrue, assertFalse, assertNull, assertNotNull, assertSame, assertNotSame, assertEquals, assertThrows, assertAll, assertTimeout, fail. Для чисел с плавающей точкой нужен delta/epsilon.

1.3 Make и Makefile

Что такое дефолтная цель в Makefile? Цель, которую make строит при запуске без аргументов. Обычно это первая цель первого правила, часто all. В GNU Make её можно переопределить специальной переменной .DEFAULT_GOAL.

Какие встроенные переменные Make знаете? Программные: CC, CXX, CPP, AS, AR, LD, RM. Флаги: CFLAGS, CXXFLAGS, CPPFLAGS, LDFLAGS, LDLIBS. Служебные: MAKE, MAKEFLAGS, MAKELEVEL, MAKECMDGOALS, CURDIR, SHELL. Есть автоматические переменные, вычисляемые для конкретного правила.

Объясните автоматические переменные `$$`, `$<`, `$$@` — — — `.$<` — — — `.$^` — — — `.` :
`$(CC) -o $$ $^`.

1.4 GNU Autotools

Какие утилиты входят в Autotools? Основные пакеты: Autoconf, Automake, Libtool. Часто встречающиеся программы: autoscan, autoconf, autoheader, autoreconf, aclocal, automake, libtoolize; под капотом используется m4/autom4te.

Какие роли у этих утилит? Autoconf генерирует переносимый скрипт configure; Automake генерирует переносимые Makefile.in из Makefile.am; Libtool упрощает сборку переносимых библиотек; autoheader создаёт шаблон config.h.in; autoscan помогает получить черновик configure.ac; aclocal собирает макросы; autoreconf запускает нужную цепочку автоматическ

1.5 Ant

Что такое Ant, как он работает? Apache Ant -- императивная система сборки для Java-проектов. Сборка описывается в build.xml: project содержит цели target, цели имеют зависимости и внутри себя выполняют задачи task. В отличие от Make, Ant использует XML-задачи вместо платформозависимых shell-команд.

Какая дефолтная цель в Ant? Цель из атрибута default тега <project>. Если запустить ant без указания цели, Ant ищет build.xml и запускает project default="...".

Что делают флаги dry-run, preview, debug, verbose? В стандартной CLI-документации Apache Ant есть -verbose/-v и -debug/-d: первый даёт больше логов, второй -- ещё более подробную отладочную информацию. dry-run и preview не являются штатными флагами Apache Ant: если их спрашивают как общий термин build tools, то это режимы “показать, что было бы выполнено, но не выполнять”. В Ant ближайшие стандартные вещи: -projecthelp для просмотра целей, подробные логи -v/-d, либо специально написанная preview-цель в build.xml.

Объясните antcall и subant. antcall вызывает другую цель внутри того же build.xml, при необходимости передавая параметры. subant запускает цели в наборе под-проектов или под-каталогов, то есть удобен для bulk-сборки модулей.

1.6 Maven

Чем Maven лучше Ant? Maven декларативнее: разработчик описывает проект в pom.xml, а не расписывает все команды сборки вручную. Maven даёт стандартную структуру каталогов, жизненный цикл, плагины, локальный и удалённые репозитории, автоматическое скачивание и транзитивное разрешение зависимостей. За это он платит меньшей гибкостью: нестандартная сборка часто требует настройки или написания плагина.

Что такое GAV? GAV -- координаты Maven-артефакта: groupId:artifactId:version. Это адрес конкретной версии компонента в репозитории.

Какие существуют scope зависимостей в Maven? compile -- по умолчанию, доступна при компиляции, тестах и runtime, транзитивна. provided -- нужна для компиляции/тестов, но в runtime её даст JDK или контейнер. runtime -- не нужна для компиляции, нужна при запуске. test -- только для тестов. system -- локальный JAR по абсолютному пути, Maven не скачивает его из репозитория. import -- только для POM в dependencyManagement, импортирует BOM.

1.7 Gradle

Чем Gradle отличается от Maven? Maven строится вокруг POM, фиксированного жизненного цикла и XML-конфигурации. Gradle строится вокруг графа задач, плагинов и DSL на Groovy/Kotlin. Он сохраняет “convention over configuration”, но даёт больше программируемости и гибкости.

Какие преимущества Gradle? Гибкий DSL, удобная кастомизация задач, поддержка Maven/Ivy-репозиторий, инкрементальная сборка, build cache, Gradle Daemon, параллельное выполнение независимых задач, хорошая поддержка multi-project builds, нейтральность к языкам и широкое применение в Android.

2 Блок 1. Основы тестирования

2.1 Термины: ошибка, дефект, отказ

В лекции используется терминология ISTQB:

- Mistake/Error -- человеческая ошибка: разработчик, аналитик или тестировщик неверно понял требование, написал неправильное условие, выбрал неверный алгоритм.
- Fault/Defect -- дефект в артефакте ПО: неправильная строка кода, неверная спецификация, ошибка в архитектуре, некорректный интерфейс.
- Failure -- наблюдаемое внешнее проявление дефекта: программа падает, возвращает неверный результат, зависает, нарушает протокол.
- Bug -- бытовой термин без строгой границы: им могут назвать и дефект, и отказ, и ошибку в требованиях.

Важно различать дефект и отказ. Дефект может долго не проявляться, если не возникает подходящая комбинация входных данных, состояния и окружения. Поэтому тестирование не доказывает отсутствие дефектов; оно показывает их наличие и повышает уверенность в качестве.

2.2 Цели тестирования

В лекции перечислены основные цели:

- обнаружение дефектов;
- повышение уверенности в уровне качества;
- предоставление информации для принятия решений;
- предотвращение дефектов.

Более практичная формулировка: тестирование повышает приемлемый уровень доверия пользователя и заказчика к тому, что программа корректно работает в нужных обстоятельствах. Ключевые слова здесь:

- корректно -- соответствует требованиям, стандартам, ожиданиям рынка и предметной области;
- доверие -- подкреплено измеримыми фактами: тестами, отчётами, метриками, динамикой дефектов;
- нужные обстоятельства -- среда должна быть похожа на реальную: нагрузка, ОС, браузеры, оборудование, данные, роли пользователей.

2.3 Верификация и валидация

Верификация отвечает на вопрос: “правильно ли мы реализовали сформулированные требования”. Это проверка соответствия реализации спецификации.

Валидация отвечает на вопрос: “те ли требования мы выбрали и тот ли продукт нужен пользователю?”. Это проверка пригодности продукта и требований для реальной задачи.

В лекции это связывается с V-моделью:

- требования проверяются через системные и приёмочные испытания;
- архитектура -- через интеграционные проверки;
- детальное проектирование и модули -- через модульные тесты;
- заказчик окончательно подтверждает пригодность продукта на приёмке.

2.4 Как проверять соответствие реализации требованиям

Нормальный процесс проверки требований выглядит так:

1. Разбить требования на проверяемые утверждения. Плохое требование: “система должна быть удобной”. Хорошее: “страница поиска должна возвращать первые результаты не позже чем за 2 секунды при 100 одновременных пользователях”.
2. Для каждого требования определить критерий приёма: входные данные, предусловия, ожидаемый результат, допустимые отклонения.
3. Построить матрицу трассируемости: требование → тестовые случаи → результаты прогонов → дефекты.
4. Выполнить статическую проверку: ревью требований, архитектуры, интерфейсов и тестовых сценариев.
5. Выполнить динамическую проверку: запустить тесты и сравнить фактический результат с ожидаемым.
6. Проверить негативные сценарии: неверные данные, ошибки пользователя, сбои внешних сервисов, недостаток ресурсов.
7. Зафиксировать результат: passed/failed/blocked, окружение, версия сборки, логи, дефекты.

2.5 Тестовый случай и тестовый сценарий

Тестовый случай -- минимальная проверка конкретного поведения. Он содержит:

- входные значения или управляющие воздействия;
- предусловия;
- шаги выполнения;
- ожидаемый результат;
- постусловия;
- фактический результат после выполнения.

Ожидаемый результат должен быть определён до запуска теста. Иначе тестирующий рискует принять правдоподобный, но ошибочный результат за корректный.

Тестовый сценарий -- последовательность тестовых случаев, обычно описывающая типичное или альтернативное использование системы. Пример из лекции: сценарий снятия денег в банкомате состоит из случаев “вставить карту”, “ввести PIN”, “выбрать операцию”, “получить деньги”. Негативный сценарий: трижды ввести неверный PIN и проверить блокировку карты.

Хороший тест повторяем: одинаковые входные данные и одинаковое состояние должны давать одинаковый результат. Это важно для автоматизации и регрессионного тестирования.

2.6 Статическое и динамическое тестирование

Статическое тестирование не требует запуска кода. Проверяются требования, дизайн, код, тест-планы, тестовые сценарии, документация. Формы:

- неформальное ревью;
- walkthrough/сквозной контроль;
- технический анализ;
- инспекция;
- автоматический статический анализ, например Lint, FindBugs/SpotBugs.

Преимущество: дефекты можно найти рано, когда исправление дешевле. Статические техники особенно хорошо находят противоречивые требования, нарушения стандартов, плохую сопровождаемость, ошибки интерфейсов, неинициализированные переменные, забытые fclose, переполнения буфера, проблемы переносимости и копипасту.

Динамическое тестирование запускает код: модуль, группу модулей или систему целиком. Оно проверяет фактическое поведение продукта на входных данных.

2.7 Ручное, автоматизированное и регрессионное тестирование

Ручное тестирование дешевле, когда сценариев мало, интерфейс часто меняется или создание автотеста дороже самой проверки.

Автоматизированное тестирование выгодно, когда проверки повторяются много раз: модульные тесты, регрессия, совместимость, smoke-тесты, CI.

Регрессионное тестирование -- повторный запуск старых тестов после изменения программы. Цель -- убедиться, что новая правка не сломала уже работавшее поведение.

Главная трудность автоматизации: тест должен быть воспроизводимым. Если система зависит от времени, случайности, внешнего сервиса или состояния базы, это состояние нужно контролировать.

3 Блок 2. Покрытие тестами

3.1 Что такое тестовое покрытие

Тестовое покрытие показывает, какая часть требований, функциональности, сценариев, состояний или кода проверена тестами. В лекции подчеркнута ключевая идея: полное покрытие практически недостижимо. Даже простая функция умножения двух 32-битных чисел имеет 2^{64} комбинаций входов, что делает полный перебор бессмысленным.

Поэтому тестирование всегда ищет баланс:

- больше тестов -- выше покрытие и качество, но выше стоимость и дольше вывод на рынок;
- меньше тестов -- быстрее выпуск, но выше риск пропустить дефекты;
- разумный компромисс выбирается по требованиям, рискам, критичности и стоимости.

3.2 Покрытие по методу чёрного ящика

Метод чёрного ящика строит тесты по внешнему описанию поведения: требованиям, спецификациям, дизайну, пользовательским сценариям, знаниям предметной области. Код внутри не анализируется.

3.2.1 Эквивалентное разбиение

Множество входных данных делится на классы, внутри которых система, как предполагается, ведёт себя одинаково. Из каждого класса выбирают несколько представителей.

Пример: возраст пользователя:

- меньше 0 -- некорректный ввод;
- 0-17 -- несовершеннолетний;
- 18-64 -- взрослый;
- 65 и больше -- льготная категория.

Тестировать все значения не нужно; достаточно представителей классов плюс границы.

3.2.2 Анализ граничных значений

Ошибки часто возникают на границах интервалов: < вместо <=, неверный индекс, переполнение off-by-one. Поэтому отдельно проверяют значения около границ: перед границей, на границе и после границы.

Для диапазона 18-64 полезные значения: 17, 18, 19, 63, 64, 65.

3.2.3 Таблицы решений

Таблица решений задаёт комбинации условий и ожидаемые действия. Она полезна, когда результат зависит от нескольких признаков: роль пользователя, статус заказа, наличие оплаты, остаток на складе.

Таблица помогает увидеть:

- пропущенные комбинации;
- противоречивые правила;
- избыточные условия;
- минимальный набор тестов для покрытия бизнес-логики.

3.2.4 Таблицы состояний и переходов

Подходит для систем с явными состояниями: банкомат, заказ, сессия пользователя, документооборот. Нужно покрыть:

- допустимые переходы;
- недопустимые переходы;
- корректные сообщения об ошибке;
- начальные и конечные состояния;
- циклы и повторные действия.

3.2.5 Сценарии использования

Use cases дают реальные пользовательские маршруты. По ним строят группы тестовых сценариев:

- основной успешный путь;
- альтернативные успешные пути;
- негативные пути;
- восстановление после ошибки.

3.3 Покрытие по методу белого ящика

Метод белого ящика анализирует код и структуру программы. По лекции код можно представить как граф: узлы -- участки кода, рёбра -- переходы, ветвления и циклы. Затем тесты выбирают так, чтобы пройти нужные элементы графа.

Основные виды:

- покрытие операторов/строк -- каждая строка или оператор выполнены хотя бы один раз;
- покрытие ветвей/решений -- каждая ветка if/else, switch, условного перехода выполнена;
- покрытие условий -- каждое атомарное условие принимало true и false;
- покрытие путей -- проверяются разные пути по графу выполнения;
- покрытие циклов -- 0 итераций, 1 итерация, несколько итераций, границы;
- MC/DC -- каждое атомарное условие должно независимо влиять на итоговое решение; применяется в критичных системах.

Высокое покрытие кода не гарантирует хорошего тестирования. Тест может выполнить строку, но ничего осмысленного не проверить. Поэтому покрытие -- метрика полноты, а не качества assertions.

3.4 Дополнительные виды покрытия

- Покрытие требований: у каждого требования есть хотя бы один тест.
- Покрытие рисков: критичные и вероятные риски проверены раньше и глубже.
- Покрытие конфигураций: ОС, браузеры, JVM, устройства, базы данных.
- Покрытие данных: типичные, граничные, пустые, некорректные, большие данные.
- Покрытие ролей: разные права пользователя и разные бизнес-роли.
- Покрытие нефункциональных требований: производительность, безопасность, надёжность, совместимость, удобство.

4 Блок 3. Уровни тестирования и V-модель

4.1 Модульное тестирование

Модульное тестирование проверяет отдельный модуль: метод, класс, компонент или небольшую группу функций. Модуль должен выполнять законченную функцию и быть изолирован от окружения.

Для изоляции используют:

- драйвер -- тестовый код, который вызывает модуль вместо реального вызывающего компонента;
- заглушку -- упрощённую замену подчинённого модуля с тем же интерфейсом;
- mock/fake/stub -- современные разновидности тестовых двойников.

Цель -- быстро локализовать дефект. Если упал модульный тест, область поиска обычно мала.

4.2 Интеграционное тестирование

Интеграционное тестирование проверяет взаимодействие модулей или систем:

- вызовы API;
- сообщения между объектами;
- работа с базой данных;
- GUI как слой взаимодействия;
- сетевые, аппаратные и локальные интерфейсы;
- взаимодействие с инфраструктурой.

В лекции выделяются две формы:

- компонентное интеграционное -- взаимодействие компонентов внутри продукта;
- системное интеграционное -- взаимодействие системы с другими системами, аппаратурой и внешними сервисами.

4.3 Стратегии интеграции

Сверху вниз. Сначала бизнес-логика и верхние уровни, нижние уровни заменяются заглушками. Плюс: быстро появляется демонстрируемое приложение. Минус: много заглушек.

Снизу вверх. Сначала нижние уровни, драйверы вызывают их вместо верхней логики. Часто полезно в аппаратно-зависимых проектах. Минус: UI и бизнес-сценарии появляются позже.

Функциональная стратегия/end-to-end. Система наращивается по пользовательским сценариям: сначала один полный путь UI-логика-данные, затем второй и так далее.

Backbone/ядро. Сначала минимальный работоспособный каркас, затем добавляются функции.

Big bang. Всё собирается сразу. Это просто, но плохо локализует дефекты: если сломалось, неясно, какой модуль или интерфейс виноват.

4.4 Функциональное тестирование

Функциональное тестирование проверяет заявленные функции системы, часто через пользовательский интерфейс и бизнес-сценарии. Оно может выполняться вручную или автоматически. Примеры инструментов автоматизации UI: Selenium, Sahi, Watir, коммерческие инструменты HP/Rational.

Особенность: полностью избежать ручной проверки трудно, особенно когда меняется интерфейс или бизнес-процесс.

4.5 Системное и приёмочное тестирование

Системное тестирование начинается после интеграции и проверяет систему целиком. Обычно идёт от простых сценариев к сложным:

1. заявленные возможности;
2. стабильность при многопользовательской работе;
3. устойчивость к сбоям и неверным данным;
4. совместимость;
5. производительность.

Альфа-тестирование выполняется пользователями под контролем разработчика, обычно в окружении разработчика.

Бета-тестирование выполняется пользователями в более реальном пользовательском окружении, но всё ещё под контролем разработчика.

Приёмочное тестирование выполняется заказчиком/пользователем в его окружении. Его итог -- решение, принимать продукт или нет.

4.6 Нефункциональное и нагрузочное тестирование

В лекции приведён набор характеристик CARAT:

- Capacity -- объём и нефункциональные возможности;
- Accuracy -- точность;
- Response Time -- время ответа;
- Availability -- готовность;
- Throughput -- пропускная способность.

Инструменты нагрузочного тестирования: HP LoadRunner, LoadComplete, IBM Rational Performance Tester, LoadUI Pro, Apache JMeter, The Grinder, Tsung. Для Java-проектов в лекции особенно выделен Apache JMeter.

4.7 V-модель

V-модель показывает соответствие между разработкой и проверкой:

Стадия разработки	Соответствующее тестирование
Бизнес-требования, требования заказчика	Приёмочное тестирование
Требования к ПО	Системное тестирование
Архитектурное проектирование, интерфейсы	Интеграционное тестирование
Детальное проектирование, модули	Модульное тестирование
Кодирование	Запуск модульных и последующих тестов

Смысл V-модели не в том, что тесты появляются только после кода. Наоборот: тестовые уровни можно планировать ещё на соответствующих стадиях левой ветви, а выполнять после реализации.

5 Блок 4. JUnit

5.1 Идея JUnit

JUnit -- фреймворк для автоматического модульного тестирования Java-кода. В лекции рассматривается JUnit 4: тестовые методы помечаются аннотациями, JUnit через reflection находит эти методы, запускает их, проверяет assertions и формирует отчёт.

Тесты в JUnit должны быть независимыми. Нельзя полагаться на порядок выполнения, если он не задан явно. Иначе один тест может проходить только потому, что предыдущий оставил нужное состояние.

5.2 Основные аннотации JUnit 4

- `@Test` -- метод является тестом.
- `@Test(expected = SomeException.class)` -- в JUnit 4 старый способ проверить ожидаемое исключение.
- `@Test(timeout = ...)` -- тест должен завершиться за заданное время.
- `@Before` -- выполняется перед каждым тестовым методом.
- `@After` -- выполняется после каждого тестового метода.
- `@BeforeClass` -- выполняется один раз перед всеми тестами класса; метод должен быть `static`.
- `@AfterClass` -- выполняется один раз после всех тестов класса; метод должен быть `static`.
- `@Ignore` -- временно отключает тест или класс.
- `@RunWith` -- задаёт кастомный runner, например параметризованный запуск.
- `@Rule` и `@ClassRule` -- правила для временных файлов, ожидаемых исключений, ресурсов и т.п.
- `@FixMethodOrder` -- явное управление порядком тестов, но обычно лучше делать тесты независимыми.

5.3 Аналоги в JUnit Jupiter

JUnit 4	JUnit Jupiter	Смысл
<code>@Before</code>	<code>@BeforeEach</code>	Перед каждым тестом
<code>@After</code>	<code>@AfterEach</code>	После каждого теста
<code>@BeforeClass</code>	<code>@BeforeAll</code>	Перед всеми тестами класса
<code>@AfterClass</code>	<code>@AfterAll</code>	После всех тестов класса
<code>@Ignore</code>	<code>@Disabled</code>	Отключить тест
<code>@RunWith</code>	<code>@ExtendWith</code>	Подключить расширение
<code>@Category</code>	<code>@Tag</code>	Пометить тест группой

Дополнительно в JUnit Jupiter часто используют `@ParameterizedTest`, `@RepeatedTest`, `@Nested`, `@Timeout`, `@TempDir`.

5.4 Assert-методы

- `assertEquals(expected, actual)` -- значения равны.
- `assertNotEquals(unexpected, actual)` -- значения не равны.
- `assertTrue(condition)` и `assertFalse(condition)` -- булево условие.
- `assertNull(obj)` и `assertNotNull(obj)` -- проверка на null.
- `assertSame(expected, actual)` -- ссылки указывают на один объект.

- `assertNotSame(unexpected, actual)` -- ссылки разные.
- `assertArrayEquals(expected, actual)` -- массивы равны по элементам.
- `assertThrows(Exception.class, executable)` -- код бросает ожидаемое исключение.
- `assertAll(...)` -- сгруппировать несколько проверок и увидеть все падения группы.
- `assertTimeout(...)` -- проверка ограничения по времени.
- `fail()` -- явно провалить тест.

Для `double/float` обычно используют сравнение с погрешностью: `assertEquals(expected, actual, delta)`.

5.5 Минимальный пример

```
import static org.junit.Assert.*;
import org.junit.Before;
import org.junit.Test;

public class CalculatorTest {
    private Calculator calculator;

    @Before
    public void setUp() {
        calculator = new Calculator();
    }

    @Test
    public void addReturnsSum() {
        assertEquals(5, calculator.add(2, 3));
    }

    @Test
    public void divideByZeroThrows() {
        assertThrows(ArithmeticException.class, () -> calculator.divide(1, 0));
    }
}
```

6 Блок 5. Зачем нужны системы сборки

6.1 Проблемы ручной сборки

Лекция начинает блок сборки с простого наблюдения: даже маленький Java-проект требует повторять цепочку команд: компиляция, копирование ресурсов, упаковка, создание manifest, запуск тестов. Вручную это плохо масштабируется.

Основные проблемы:

- команды повторяются и легко ошибиться;
- проект меняется, скрипты устаревают;
- есть внешние зависимости;
- среда разработки отличается от среды выполнения;
- разные ОС, библиотеки, пути и архитектуры;
- долго компилировать всё с нуля;
- нужен учёт времён модификации и зависимостей;
- нужно распараллеливать работу;
- CI должен собирать продукт автоматически.

6.2 Что делают build tools

Современные средства сборки:

- описывают процесс сборки специальным языком;
- строят граф зависимостей;
- пересобирают только устаревшие части;
- позволяют задавать конфигурацию и платформу;
- умеют работать с репозиториями зависимостей;
- интегрируются с тестами и CI;
- поддерживают параллельную и иногда распределённую сборку;
- создают артефакты: JAR, WAR, ZIP, бинарники, отчёты.

6.3 Императивный и декларативный подход

Императивная сборка: разработчик описывает, как именно строить продукт, шаг за шагом. Примеры: Make, Ant. Плюс -- гибкость. Минус -- много ручной логики.

Декларативная сборка: разработчик описывает, что за проект и какой результат нужен, а инструмент сам выбирает стандартные шаги. Пример: Maven. Плюс -- меньше ручной работы и единые правила. Минус -- нестандартные сценарии сложнее.

Гибридный подход: есть соглашения и плагины, но логика сборки программируема. Пример: Gradle.

7 Блок 6. Make и Makefile

7.1 Идея Make

Make -- низкоуровневое средство сборки из Unix-экосистемы. Оно читает Makefile, строит цели с учётом зависимостей и времён модификации файлов.

Правило имеет вид:

```
target: prerequisites
    recipe
```

- target -- цель: файл или условное действие, например clean;
- prerequisites -- зависимости, необходимые для построения цели;
- recipe -- команды shell, которые выполняются для цели;
- строка recipe обычно начинается с tab.

Если цель старше любой зависимости или цель отсутствует, Make выполняет recipe.

7.2 Дефолтная цель

Если запустить просто make, GNU Make берёт первую цель первого правила. Обычно первой пишут цель all, чтобы она собирала весь проект:

```
.DEFAULT_GOAL := all

all: app tests

app: main.o util.o
    $(CC) -o $@ $^
```

Технически дефолтную цель можно задать через .DEFAULT_GOAL; но хорошая практика -- делать первую цель очевидной.

7.3 Обычные переменные Make

Переменная	Смысл
CC	C-компилятор, часто cc или gcc
CXX	C++-компилятор
CPP	C-препроцессор
AS	Ассемблер
AR	Архиватор статических библиотек
LD	Линкер
RM	Удаление файлов
CFLAGS	Флаги C-компилятора: оптимизация, warnings
CXXFLAGS	Флаги C++-компилятора
CPPFLAGS	Флаги препроцессора: include paths, defines
LDLFLAGS	Флаги линковщика: пути библиотек, режимы линковки
LDLIBS	Библиотеки для линковки, например -lm
MAKE	Команда текущего Make, полезна для рекурсивного запуска
MAKEFLAGS	Флаги, переданные Make
MAKELEVEL	Глубина рекурсивного Make
MAKECMDGOALS	Цели, указанные в командной строке
CURDIR	Текущий каталог Make
SHELL	Shell для выполнения recipe

7.4 Автоматические переменные

Автоматические переменные вычисляются для конкретного правила и имеют смысл только внутри recipe.

Переменная	Смысл
\$@	Имя цели текущего правила
\$<	Первая зависимость
\$^	Все зависимости без дублей
\$?	Зависимости, которые новее цели
\$+	Все зависимости с сохранением дублей
\$*	Stem в pattern rule, часть имени, соответствующая %
\$(@D), \$(@F)	Каталог и имя файла цели

Пример:

```
main.o: main.c main.h
    $(CC) $(CPPFLAGS) $(CFLAGS) -c $< -o $@

app: main.o util.o
    $(CC) $(LDLFLAGS) -o $@ $^ $(LDLIBS)
```

7.5 Плюсы и минусы Make

Плюсы: простота идеи, универсальность, работает почти с любыми командами, инкрементальная сборка по timestamps, много встроенных правил.

Минусы: сложный синтаксис, tab-ловушки, много неявного поведения, зависимость от shell и ОС, трудно поддерживать большие проекты без генераторов.

8 Блок 7. GNU Autotools

8.1 Назначение

Autotools решают проблему переносимой сборки, особенно для C/C++ и Unix-подобных систем. Их задача -- сгенерировать корректный `configure` и `Makefile`, учитывая:

- операционную систему;
- компилятор;
- наличие заголовков и библиотек;
- наличие функций в системных библиотеках;
- пользовательские опции конфигурации;
- пути установки.

Для пользователя классическая цепочка проста:

```
./configure
make
sudo make install
```

8.2 Основные утилиты и роли

Утилита	Роль
m4, autom4te	Макропроцессорная основа Autoconf
autoscan	Сканирует исходники и создаёт черновик <code>configure.scan</code>
autoconf	Из <code>configure.ac</code> генерирует скрипт <code>configure</code>
autoheader	Генерирует <code>config.h.in</code>
aclocal	Собирает m4-макросы, нужные Automake/Autoconf
automake	Из <code>Makefile.am</code> генерирует <code>Makefile.in</code>
autoreconf	Запускает нужные autotools-команды автоматически
libtoolize	Подготавливает поддержку Libtool
libtool	Помогает переносимо собирать shared/static libraries

8.3 Файлы проекта

`configure.ac` описывает проект и проверки:

- имя и версию проекта;
- где лежат исходники;
- какие компиляторы нужны;
- какие заголовки, библиотеки и функции проверить;
- какие файлы сгенерировать.

Типичные макросы: `AC_INIT`, `AC_CONFIG_SRCDIR`, `AC_CONFIG_HEADERS`, `AC_PROG_CC`, `AC_CHECK_HEADERS`, `PKG_CHECK_MODULES`.

`Makefile.am` описывает, какие программы и библиотеки строить, из каких исходников и с какими флагами. Automake превращает его в переносимый `Makefile.in`.

8.4 Процесс для разработчика

1. Написать исходный код.
2. Запустить `autoscan`, получить `configure.scan`.
3. Отредактировать его в `configure.ac`.

4. Написать `Makefile.am`.
5. Запустить `aclocal`, `autoheader`, `automake`, `autoconf` или просто `autoreconf -i`.
6. Получить `configure`, `Makefile.in`, `config.h.in`.
7. Распространять исходники так, чтобы пользователь мог выполнить `./configure` и `make`.

8.5 Что делает `./configure`

`./configure` запускается уже на целевой машине. Он:

- проверяет компилятор;
- компилирует маленькие тестовые программы;
- проверяет функции и библиотеки;
- создаёт `config.h`;
- создаёт `Makefile` из `Makefile.in`;
- учитывает опции вроде `-prefix=/usr/local`.

После этого обычный `make` работает с уже настроенным `Makefile`.

8.6 Плюсы и минусы Autotools

Плюсы: переносимость, зрелость, привычный интерфейс для open-source, мощные проверки системы, стандартные цели `make`, `make install`, `make check`.

Минусы: сложность, много сгенерированных файлов, `m4`-синтаксис, высокий порог входа, часть проблем проще решается современными системами вроде `CMake`/`Meson`.

9 Блок 8. Apache Ant

9.1 Идея Ant

Apache Ant -- императивный Java build tool. Он похож на Make тем, что описывает цели и зависимости, но вместо shell-команд использует XML-задачи. Это уменьшает зависимость от ОС.

Минимальная структура:

```
<project name="World" default="dist" basedir=".">
  <target name="init">
    <mkdir dir="build"/>
  </target>

  <target name="compile" depends="init">
    <javac srcdir="src" destdir="build"/>
  </target>

  <target name="dist" depends="compile">
    <jar destfile="app.jar" basedir="build"/>
  </target>
</project>
```

9.2 Project, target, task

- project -- весь проект сборки; задаёт имя, базовый каталог, default target.
- target -- именованная цель, например compile, test, dist.
- depends -- зависимости между целями.
- task -- конкретное действие внутри цели: javac, copy, jar, junit, delete.

Ant выполняет цели по графу зависимостей. Если указать ant dist, он сначала выполнит цели, от которых зависит dist.

9.3 Default target

Если команда ant вызвана без цели, Ant:

1. ищет build.xml в текущем каталоге;
2. читает <project default="...">;
3. запускает цель, указанную в default.

Если default не задан, сборка без явной цели не имеет очевидного входа.

9.4 Свойства Ant

Свойства похожи на переменные, но в Ant они фактически неизменяемы после первого присваивания. Их можно задавать:

- напрямую: <property name="build.dir" value="build"/>;
- из окружения: <property environment="env"/>;
- из файла: <property file="build.properties"/>;
- из командной строки: ant -Dbuild.dir=out.

Подстановка: \${build.dir}.

9.5 Типичные задачи Ant

- файловые: copy, delete, mkdir, move, chmod;
- архивы: jar, war, ear, zip, tar, gzip;
- Java: javac, java, javadoc, rmic;
- тесты: junit;
- управление потоком: parallel, sequential, waitfor, sleep;
- внешние команды: exec;
- SCM: cvs, vss, clearcase.

9.6 CLI-флаги

Стандартные флаги Apache Ant:

- -help/-h -- помощь;
- -projecthelp/-p -- список целей проекта и default target;
- -version -- версия Ant;
- -diagnostics -- диагностическая информация;
- -quiet/-q -- меньше вывода;
- -silent/-S -- почти ничего, кроме вывода задач и ошибок сборки;
- -verbose/-v -- подробный вывод;
- -debug/-d -- отладочный вывод, больше чем verbose;
- -buildfile/-file/-f -- другой build file;
- -Dname=value -- свойство из командной строки;
- -keep-going/-k -- продолжать независимые цели после ошибки.

Про dry-run/preview. В официальной CLI-документации Ant эти флаги не указаны. На экзамене лучше ответить так: “в Ant штатно есть -v/-d и -p; dry-run или preview как общий термин означает предварительный просмотр действий без выполнения, но в чистом Ant это обычно реализуется через отдельную цель, logger, IDE или wrapper, а не стандартным флагом”.

9.7 antcall

antcall вызывает другую цель того же build.xml не через обычную зависимость, а явно из текущей цели. Можно передать параметры:

```
<target name="dist">
  <antcall target="compile">
    <param name="debug" value="true"/>
  </antcall>
</target>
```

Нюанс: вызванная цель запускается в новом Ant project context. Свойства, заданные внутри вызванной цели, не возвращаются назад в вызывающий target.

9.8 subant

subant запускает Ant-сборку для набора подпроектов. Это удобно для модульной структуры:

```
<target name="compile-all">
  <subant target="compile">
    <fileset dir="modules" includes="*/build.xml"/>
  </subant>
</target>
```

Главное отличие: `antcall` -- цель в этом же файле; `subant` -- много сборок в других каталогах/подпроектах.

9.9 Ivy

В лекции Ivy упоминается как менеджер зависимостей для Ant. До Ivy зависимости часто приходилось скачивать и подключать вручную. Ivy умеет работать с Maven-репозиториями и описывает зависимости в `ivy.xml`.

10 Блок 9. Maven

10.1 Идея Maven

Maven -- декларативная система сборки, особенно популярная в Java. Главный файл -- `pom.xml`, Project Object Model. POM описывает:

- имя, версию и тип проекта;
- координаты артефакта;
- зависимости;
- плагины;
- профили;
- репозитории;
- дополнительную метаинформацию.

В отличие от Ant, Maven не требует подробно описывать каждое действие. Он применяет стандартный жизненный цикл и плагины.

10.2 Чем Maven лучше Ant

- Стандартная структура проекта: `src/main/java`, `src/test/java`, `src/main/resources`, `target`.
- Жизненный цикл: `validate`, `compile`, `test`, `package`, `verify`, `install`, `deploy`.
- Dependency management: зависимости описываются декларативно и скачиваются автоматически.
- Транзитивные зависимости: Maven подтягивает зависимости зависимостей.
- Репозитории: локальный `~/.m2/repository`, Maven Central, корпоративные репозитории.
- Плагины: компиляция, тесты, упаковка, отчёты, деплой выполняются `plugin goals`.
- Единообразие: новый разработчик быстрее понимает проект.

Минус Maven: если проект выходит за рамки стандартного жизненного цикла, настройка может быть сложной, а иногда приходится писать собственный плагин.

10.3 GAV

GAV -- это `groupId:artifactId:version`.

- `groupId` -- организация или группа проекта, например `org.junit.jupiter`;
- `artifactId` -- имя артефакта, например `junit-jupiter-api`;
- `version` -- конкретная версия.

В репозитории это превращается в путь. Условно:

```
groupId/artifactId/version/artifactId-version.jar
```

Например:

```
<dependency>
  <groupId>junit</groupId>
  <artifactId>junit</artifactId>
  <version>4.13.2</version>
  <scope>test</scope>
</dependency>
```


10.4 Scope зависимостей

Scope	Смысл
compile	Scope по умолчанию. Зависимость нужна для компиляции, тестов и запуска. Транзитивно передаётся зависимым проектам.
provided	Нужна для компиляции и тестов, но при запуске её предоставит JDK, сервер приложений или контейнер. Пример: Servlet API в WAR-приложении. Не транзитивна.
runtime	Для компиляции не нужна, но нужна при запуске. Попадает в runtime и test classpath.
test	Только для компиляции и запуска тестов. Не нужна обычному приложению. Примеры: JUnit, Mockito. Не транзитивна.
system	Похожа на provided, но JAR задаётся локальным абсолютным путём через systemPath; Maven не скачивает его из репозитория. Обычно нежелательна.
import	Только для dependency типа pom в dependencyManagement. Используется для BOM и импортирует набор управляемых зависимостей.

10.5 Жизненный цикл Maven

Основные фазы default lifecycle:

- validate -- проверить корректность проекта;
- compile -- скомпилировать основной код;
- test -- запустить unit-тесты;
- package -- упаковать результат в JAR/WAR/EAR;
- verify -- выполнить проверки качества, включая интеграционные тесты;
- install -- положить артефакт в локальный репозиторий;
- deploy -- отправить артефакт в удалённый репозиторий.

Если вызвать `mvn package`, Maven выполнит все предыдущие фазы до `package`. Если вызвать `mvn install`, он дойдёт до установки в локальный репозиторий.

Оговорка по лекции: в ней сказано, что цель Maven по умолчанию -- `install`. В текущей практике Maven фазу/goal обычно указывают явно: `mvn test`, `mvn package`, `mvn install`, `mvn verify`. Без цели Maven обычно не выполняет “магический” `install`; возможны настройки `defaultGoal` в POM.

10.6 Плагины

Maven-фазы сами по себе абстрактны. Реальную работу делают плагины и их goals:

- `compiler:compile` -- компиляция;
- `surefire:test` -- unit-тесты;
- `failsafe:integration-test/verify` -- интеграционные тесты;
- `jar:jar`, `war:war`, `ear:ear` -- упаковка;
- `install:install` -- локальная установка;
- `deploy:deploy` -- публикация.

10.7 Профили

Профиль -- альтернативная конфигурация проекта: debug/release, разные окружения, разные репозитории, разные плагины. Активируется, например, так:

```
mvn package -Pprod
```

11 Блок 10. Gradle

11.1 Идея Gradle

Gradle -- гибридная система сборки. Она взяла идеи Ant и Maven:

- от Ant -- программируемость, задачи, гибкость;
- от Maven -- репозитории, зависимости, плагины, соглашения;
- сверху добавила DSL на Groovy/Kotlin, task graph, инкрементальность, cache и daemon.

В лекции Gradle описан как более гибкий преемник Ant и Maven: он нейтрален к языкам, может использовать Maven/Ivy-репозитории, работает с GAV-нотацией и поддерживает инкрементальную и параллельную сборку.

11.2 Пример Gradle build script

```
plugins {
    id 'java'
    id 'application'
}

repositories {
    mavenCentral()
}

dependencies {
    implementation 'org.apache.logging.log4j:log4j-core:2.12.1'
    testImplementation 'junit:junit:4.13.2'
}

application {
    mainClass = 'ru.ifmo.cs.BcompNG'
}
```

11.3 Чем Gradle отличается от Maven

Критерий	Maven	Gradle
Описание сборки	XML POM	Groovy/Kotlin DSL
Модель	Фиксированный lifecycle	Граф задач
Подход	Декларативный	Декларативный + программируемый
Кастомизация	Через плагины и конфигурацию	Через задачи, плагины и код DSL
Производительность	Последовательное, меньше work avoidance	Incremental build, cache, daemon, parallel tasks
Зависимости	Maven repositories, scopes	Maven/Ivy repositories, configurations
Нестандартные проекты	Часто сложнее	Обычно гибче

11.4 Жизненный цикл Gradle

Gradle-сборка имеет три фазы:

1. Initialization -- Gradle определяет root project, subprojects, included builds, читает settings.gradle(.kts).
2. Configuration -- читает build scripts, регистрирует задачи, строит task graph.
3. Execution -- выполняет выбранные задачи в нужном порядке; независимые задачи могут идти параллельно.

Это отличается от Maven: в Maven разработчик выбирает фазу lifecycle, а Gradle строит граф задач и выполняет выбранные задачи с их зависимостями.

11.5 Преимущества Gradle

- Гибкость: можно описывать нестандартную сборку без написания тяжёлого Maven plugin.
- DSL: build script читается как код сборки, поддерживает условия, функции, переиспользование.
- Инкрементальная сборка: если inputs/outputs задачи не изменились, задача получает статус UP-TO-DATE и не выполняется.
- Build cache: результаты задач можно переиспользовать локально или между машинами.
- Gradle Daemon: долгоживущий процесс ускоряет повторные сборки.
- Parallel execution: независимые задачи могут выполняться параллельно.
- Multi-project builds: удобно строить большие модульные проекты.
- Wrapper: gradlew фиксирует версию Gradle для проекта.
- Language neutrality: Java, Kotlin, Groovy, Scala, Android, C++, Swift и т.д.
- Android: Gradle является стандартным build tool для Android-проектов.

11.6 Минусы Gradle

- Сложный DSL может превратиться в трудно поддерживаемый код.
- Больше свободы -- больше способов сделать плохо.
- Для больших проектов важно аккуратно описывать inputs/outputs, иначе cache и incremental build будут работать неправильно.
- Новичку Maven иногда проще из-за более жёсткой модели.

12 Приложение: лабораторная 2

Этот блок разворачивает вопросы из файла РьРерс2.pdf. Формулировки сделаны так, чтобы их можно было использовать как готовые ответы: сначала суть, затем детали, которые удобно проговорить на защите.

12.1 Тестирование ПО

Тестирование ПО -- это процесс проверки программного продукта с целью найти дефекты, оценить качество и получить информацию о том, насколько продукт соответствует требованиям и ожиданиям пользователей. Тестирование не доказывает, что дефектов нет: оно показывает, что в выбранных условиях и на выбранных данных программа ведёт себя ожидаемо.

В тестировании важно заранее определить:

- что именно проверяется: модуль, интеграция, вся система, требование, сценарий;
- на каких данных выполняется проверка;
- какой результат считается правильным;
- в каком окружении проводится проверка;
- как фиксируются результат, дефект и повторная проверка.

12.2 Цели тестирования

Основные цели:

1. Обнаружение ошибок и дефектов. Тесты должны находить места, где программа работает не так, как должна.
2. Проверка соответствия требованиям. Реализация сравнивается с техническим заданием, пользовательскими историями, спецификациями и критериями приёмки.
3. Оценка качества продукта. Тестирование даёт информацию о надёжности, производительности, удобстве, совместимости и других свойствах.
4. Снижение рисков. Чем критичнее функция, тем раньше и глубже её нужно тестировать.
5. Информация для принятия решений. По результатам тестов решают, выпускать версию, дорабатывать, откатывать изменение или переносить релиз.
6. Предотвращение повторных ошибок. Регрессионные тесты и тесты на найденные ранее дефекты не дают тем же проблемам вернуться.

12.3 Ошибка, дефект и отказ

- Ошибка -- неправильное действие человека: неверно понял требование, неправильно написал условие, выбрал неверный алгоритм.
- Дефект -- неправильность в программе или другом артефакте, появившаяся из-за ошибки: баг в коде, неверная спецификация, некорректный интерфейс.
- Отказ -- внешнее проявление дефекта при выполнении: падение программы, неправильный результат, зависание, неверное сообщение пользователю.

Пример: разработчик ошибся и написал цикл с неверной границей. В коде появился дефект. Когда пользователь вводит данные, на которых цикл пропускает нужный элемент, возникает отказ: результат неверный.

12.4 Виды тестирования

12.4.1 По уровню тестирования

- Модульное. Проверяет отдельный метод, класс, функцию или компонент. Обычно быстрое, автоматизированное, изолированное.
- Интеграционное. Проверяет взаимодействие нескольких модулей: API, сообщения, базу данных, внешний сервис, UI-слой.
- Системное. Проверяет всю систему целиком в окружении, близком к реальному.
- Приёмочное. Проверяет, готова ли система к передаче заказчику и решает ли она бизнес-задачу.

12.4.2 По способу выполнения

- Ручное. Тестировщик выполняет сценарии вручную. Полезно для exploratory testing, новых интерфейсов и нестабильных требований.
- Автоматизированное. Тесты запускаются программно. Полезно для unit-тестов, регрессии, CI, повторяемых проверок и большого количества конфигураций.

12.4.3 По знанию внутреннего устройства

- Чёрный ящик. Тестировщик знает входы, выходы и требования, но не опирается на код.
- Белый ящик. Тесты проектируются с учётом кода, ветвлений, условий, путей, циклов и структуры программы.

- Серый ящик. Есть частичное знание внутренней структуры: например, известно, какая база данных используется или какие API вызываются.

12.4.4 По цели тестирования

- Функциональное. Проверяет, выполняет ли система заявленные функции.
- Нефункциональное. Проверяет свойства, не сводящиеся к отдельной бизнес-функции: производительность, безопасность, удобство, совместимость, надёжность.
- Регрессионное. Проверяет, не сломались ли старые функции после изменений.
- Smoke-тестирование. Быстрая поверхностная проверка основных функций сборки: “система вообще жива или нет”.
- Sanity-тестирование. Узкая проверка конкретной исправленной или изменённой области.
- Нагрузочное. Проверяет поведение при ожидаемой рабочей нагрузке.
- Стресс-тестирование. Проверяет поведение выше ожидаемой нагрузки и на пределе.
- Тестирование безопасности. Проверяет устойчивость к атакам, ошибкам авторизации, утечкам данных.
- Тестирование совместимости. Проверяет работу в разных ОС, браузерах, версиях JVM, устройствах, конфигурациях.
- Тестирование удобства использования. Проверяет, насколько пользователю понятно и удобно решать задачу.

12.5 Модульное тестирование

Модульное тестирование проверяет маленькую часть программы отдельно от остальной системы. В Java это чаще всего метод или класс. Главная идея: если маленькая часть работает правильно в изоляции, то дефект проще найти и исправить.

Хороший модульный тест:

- быстрый;
- изолированный от сети, базы данных, файловой системы и других нестабильных зависимостей;
- повторяемый;
- самопроверяемый, то есть имеет assertions;
- независимый от порядка запуска других тестов;
- проверяет одно поведение, а не всё сразу.

12.6 Принцип FIRST

FIRST -- набор свойств хорошего unit-теста:

- Fast -- тест быстро выполняется. Медленные unit-тесты редко запускают.
- Independent -- тест не зависит от других тестов и их порядка.
- Repeatable -- тест даёт одинаковый результат в одинаковых условиях.
- Self-validating -- тест сам говорит pass/fail через assertions, без ручной проверки.
- Timely -- тест пишется вовремя: до кода в TDD или рядом с реализацией, пока контекст свежий.

12.7 Структура AAA

AAA -- удобная структура теста:

- Arrange -- подготовка данных, объекта, зависимостей и состояния.
- Act -- выполнение тестируемого действия.
- Assert -- проверка результата.

Пример:

```
@Test
void addReturnsSum() {
    Calculator calculator = new Calculator(); // Arrange
    int result = calculator.add(2, 3); // Act
    assertEquals(5, result); // Assert
}
```

12.8 Given–When–Then

Given–When–Then -- та же логика, но ближе к описанию поведения:

- Given -- дано начальное состояние;
- When -- когда выполняется действие;
- Then -- тогда ожидается результат.

Этот стиль часто используют в BDD и тестах бизнес-сценариев. Например: “дан пользователь с недостаточным балансом; когда он пытается оплатить заказ; тогда система отклоняет оплату и показывает причину”.

12.9 Что проверять в модульных тестах

- обычные корректные случаи;
- обычные некорректные случаи;
- граничные значения;
- пустые значения, null, пустые коллекции;
- некорректные данные;
- исключения;
- особые правила бизнес-логики;
- побочные эффекты: изменение состояния, вызов зависимости, запись события;
- идемпотентность, если операция должна давать один результат при повторном вызове.

12.10 Тестовые двойники

Тестовые двойники заменяют реальные зависимости, чтобы тест был изолированным и управляемым.

- Dummy -- объект, который передаётся только для заполнения параметра, но реально не используется.
- Stub -- объект, который возвращает заранее заданные данные.
- Fake -- упрощённая, но рабочая реализация. Например, in-memory repository вместо настоящей базы.
- Mock -- объект, на котором проверяют взаимодействие: был ли вызван метод, с какими аргументами и сколько раз.
- Spy -- частично настоящий объект, который дополнительно записывает вызовы или позволяет переопределить часть поведения.

Важно не злоупотреблять mock-объектами. Если тест проверяет слишком много внутренних вызовов, он становится хрупким: реализация меняется, поведение то же, а тест падает.

12.11 Test-Driven Development

TDD -- подход, при котором тест пишется до реализации. Классический цикл:

1. Red. Написать тест на новое поведение. Он должен не проходить, потому что реализации ещё нет или она неправильная.

2. Green. Написать минимальный код, чтобы тест прошёл.
3. Refactor. Улучшить код без изменения поведения, все тесты должны остаться зелёными.

Плюсы TDD: код получается более тестируемым, требования уточняются через примеры, регрессия появляется сразу. Минусы: требует дисциплины, не всегда удобно для UI, прототипов и плохо понятных исследовательских задач.

12.12 JUnit

JUnit -- популярный фреймворк для модульного тестирования Java-программ. В обычной речи часто говорят “JUnit 5”, имея в виду современную модель JUnit Jupiter. Архитектурно JUnit состоит из нескольких частей:

- JUnit Platform -- платформа запуска тестов на JVM; на ней работают разные test engines.
- JUnit Jupiter -- API и engine для современных тестов.
- JUnit Vintage -- модуль для запуска старых тестов JUnit 3 и JUnit 4.

12.13 Аннотации JUnit

Основные аннотации JUnit Jupiter:

- @Test -- метод является тестом.
- @BeforeEach -- выполнить перед каждым тестом.
- @AfterEach -- выполнить после каждого теста.
- @BeforeAll -- выполнить один раз перед всеми тестами класса.
- @AfterAll -- выполнить один раз после всех тестов класса.
- @Disabled -- отключить тест или класс.
- @DisplayName -- красивое имя теста в отчёте.
- @Nested -- вложенный класс тестов для группировки сценариев.
- @RepeatedTest -- повторяемый тест.
- @ParameterizedTest -- параметризованный тест с разными наборами данных.
- @Tag -- метка для фильтрации тестов.
- @Timeout -- ограничение времени выполнения.

В файле лаб2 было написано ParametrizedTest; правильное имя аннотации -- @Parameteriz

12.14 Assertions в JUnit

- assertEquals(expected, actual) -- значения равны.
- assertNotEquals(unexpected, actual) -- значения не равны.
- assertTrue(condition) -- условие истинно.
- assertFalse(condition) -- условие ложно.
- assertNull(value) -- значение равно null.
- assertNotNull(value) -- значение не равно null.
- assertSame(expected, actual) -- две ссылки указывают на один объект.
- assertNotSame(unexpected, actual) -- ссылки указывают на разные объекты.
- assertEqualsArrays(expected, actual) -- массивы равны по элементам.
- assertThrows(Exception.class, executable) -- код бросает ожидаемое исключение.
- assertAll(...) -- группа проверок, где JUnit покажет все падения группы.
- fail() -- принудительно провалить тест.

Для double и float важно сравнивать с погрешностью, а не простым равенством.

12.15 Системы автоматической сборки

Сборка -- процесс преобразования исходного кода, ресурсов и зависимостей в готовый артефакт: исполняемый файл, библиотеку, JAR, WAR, Docker image, архив или пакет для распространения.

Системы сборки нужны для:

- автоматизации компиляции;
- управления зависимостями;
- запуска тестов;
- генерации кода и ресурсов;
- упаковки приложения;
- повторяемости сборки;
- интеграции с CI/CD;
- упрощения командной разработки;
- публикации артефактов в локальные и удалённые репозитории.

12.16 Принципы работы систем сборки

Обычно build tool работает так:

1. Читает описание проекта: Makefile, build.xml, pom.xml, build.gradle.
2. Определяет задачи или фазы сборки.
3. Строит зависимости между задачами.
4. Проверяет, что нужно выполнить заново, а что уже актуально.
5. Выполняет команды, задачи или плагины.
6. Запускает тесты и формирует отчёты.
7. Создаёт итоговый артефакт.

Ключевые принципы:

- описание задач;
- зависимости между задачами;
- инкрементальность: не пересобирать неизменённое;
- стандартизация структуры проекта;
- расширяемость через плагины;
- повторяемость: одна и та же версия исходников должна давать один и тот же результат.

12.17 Основные системы сборки

- Make: файл Makefile, низкоуровневая универсальная сборка через цели, зависимости и shell-команды.
- Ant: файл build.xml, императивная XML-сборка, часто для Java.
- Maven: файл pom.xml, декларативная сборка, жизненный цикл, зависимости и плагины.
- Gradle: файлы build.gradle и settings.gradle, DSL на Groovy/Kotlin, граф задач, гибкость, инкрементальность и кэширование.

12.18 Make

Make описывает цели, зависимости и команды:

```
target: dependencies
    command
```

Смысл:

- цель -- то, что нужно получить или выполнить;

- зависимости -- файлы или цели, от которых зависит результат;
- команды -- shell-команды для построения цели;
- Make сравнивает время изменения файлов и пересобирает только устаревшие цели;
- .PHONY показывает, что цель не является файлом, например clean;
- Make позволяет автоматизировать почти любые shell-команды, не только компиляцию.

Пример:

```
.PHONY: all clean

all: app

app: main.o util.o
    $(CC) -o $@ $^

clean:
    rm -f app *.o
```

12.19 Ant

Ant использует XML-файл build.xml. В нём есть:

- project -- проект сборки;
- target -- цель сборки;
- depends -- зависимости target от других target;
- task -- конкретное действие внутри target.

Пример:

```
<project name="demo" default="jar">
  <target name="compile">
    <mkdir dir="build"/>
    <javac srcdir="src" destdir="build"/>
  </target>

  <target name="jar" depends="compile">
    <jar destfile="demo.jar" basedir="build"/>
  </target>
</project>
```

Фраза ‘таргеты зависят от таргетов’ означает, что Ant сначала выполняет цели, указанные в depends, и только потом текущую цель.

12.20 Типичные Ant tasks

- mkdir -- создать каталог;
- delete -- удалить файлы или каталоги;
- javac -- скомпилировать Java-код;
- java -- запустить Java-класс;
- junit -- запустить JUnit-тесты;
- copy -- скопировать файлы;
- jar -- собрать JAR-архив;
- echo -- вывести сообщение;
- war -- собрать web application archive;
- exec -- выполнить внешнюю команду.

12.21 Maven: что важно добавить к концу лаб2

В исходном файле Maven только назван, но для защиты стоит знать больше.

Maven описывает проект в `pom.xml`. Он работает не как “список команд”, а как декларативная модель проекта:

- координаты проекта: `groupId`, `artifactId`, `version`;
- зависимости;
- плагины;
- стандартная структура каталогов;
- жизненный цикл сборки.

Основные фазы Maven:

- `validate` -- проверить проект;
- `compile` -- скомпилировать код;
- `test` -- запустить тесты;
- `package` -- упаковать в JAR/WAR;
- `verify` -- выполнить дополнительные проверки;
- `install` -- положить артефакт в локальный репозиторий;
- `deploy` -- отправить артефакт в удалённый репозиторий.

Maven удобнее Ant для типового Java-проекта, потому что сам управляет зависимостями, использует стандартную структуру и жизненный цикл. Минус: нестандартную сборку сложнее описать.

12.22 Gradle: что важно добавить к концу лаб2

Gradle -- более гибкая система сборки, которая сочетает идеи Ant и Maven. Сборка описывается в `build.gradle` или `build.gradle.kts`, а многомодульность -- в `settings.gradle`.

Главные особенности:

- DSL на Groovy или Kotlin вместо XML;
- граф задач, а не только фиксированный lifecycle;
- поддержка Maven- и Ivy-репозитория;
- инкрементальная сборка;
- `build cache`;
- параллельное выполнение независимых задач;
- Gradle Wrapper для фиксированной версии Gradle в проекте;
- хорошая поддержка Android и multi-project builds.

Пример:

```
plugins {
    id 'java'
}

repositories {
    mavenCentral()
}

dependencies {
    testImplementation 'org.junit.jupiter:junit-jupiter:5.10.0'
}
```

Если кратко сравнить: Maven проще и стандартизированнее для типовых проектов, Gradle гибче и часто быстрее на больших сборках за счёт инкрементальности, кэша и daemon-процесса.